

Ron Vincent Cada

Student Enrollment Database – Design, Build & Query

PART 1 – Schema Design & Creation

```
create table students (
  id serial primary key,
  first_name varchar(250) not null,
  last_name varchar(250) not null,
  year int check (year >= 1 and year <= 4) not null,
  gender varchar(50),
  enrolled boolean default true,
  created_at timestampz default now()
);
```

Name	Value
Updated Rows	0
Execute time	0.024s
Start time	Mon May 11 14:57:02 PST 2026
Finish time	Mon May 11 14:57:02 PST 2026
Query	create table students (id serial primary key, first_name varchar(250) not null, last_name varchar(250) not null, year int check (year >= 1 and year <= 4) not null, gender varchar(50), enrolled boolean default true, created_at timestampz default now());

```
create table programs (
  id serial primary key,
  program_name varchar(150) not null unique
);
```

Name	Value
Updated Rows	0
Execute time	0.005s
Start time	Mon May 11 14:57:22 PST 2026
Finish time	Mon May 11 14:57:22 PST 2026
Query	create table programs (id serial primary key, program_name varchar(150) not null unique);

```
create table sections (
  id serial primary key,
  code varchar(50) not null unique,
  year int not null,
  program_id int not null references programs(id)
);
```

Name	Value
Updated Rows	0
Execute time	0.008s
Start time	Mon May 11 14:57:56 PST 2026
Finish time	Mon May 11 14:57:56 PST 2026
Query	create table sections (id serial primary key, code varchar(50) not null unique, year int not null, program_id int not null references programs(id));

```
create table student_sections (
  id serial primary key,
  student_id int not null references students(id),
  section_id int not null references sections(id),
  enrolled_at timestampz default now()
);
```

Name	Value
Updated Rows	0
Execute time	0.003s
Start time	Mon May 11 14:59:13 PST 2026
Finish time	Mon May 11 14:59:13 PST 2026
Query	create table student_sections (id serial primary key, student_id int not null references students(id), section_id int not null references sections(id), enrolled_at timestampz default now());

<pre> create table student_grades (id serial primary key, student_id int not null references students(id), grade int check (grade >= 0 and grade <= 100)) </pre>	
Statistics 1	
Name	Value
Updated Rows	0
Execute time	0.016s
Start time	Mon May 11 14:59:38 PST 2026
Finish time	Mon May 11 14:59:38 PST 2026
Query	create table student_grades (id serial primary key, student_id int not null references students(id), grade int check (grade >= 0 and grade <= 100))

PART 2 – Seed Data & INSERT

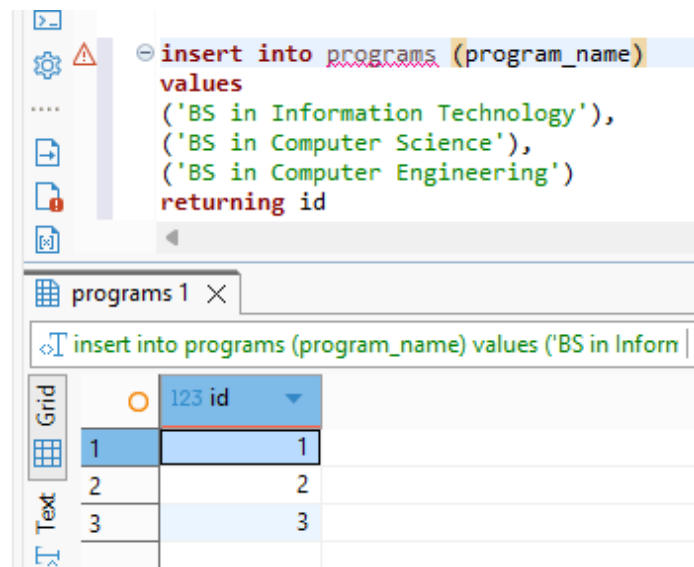
Insert data in students table

<pre> insert into students (first_name, last_name, year, gender, enrolled) values ('Ron Vincent', 'Cada', 3, 'male', true), ('Mika Andrea', 'Gomez', 2, 'female', true), ('Roosc', 'Zano', 1, 'male', false), ('Kurt', 'Laja', 3, 'male', true) returning id </pre>	
students 1	
insert into students (first_name, last_name, year, gender, e	Enter a SQL expression to filter result
Grid	123 id
1	1
2	2
3	3
4	4

Verify the count of the students table

<pre> select count(*) from students </pre>	
Results 1	
select count(*) from students	Enter a SQL expres
Grid	123 count
1	4

Insert data in programs table



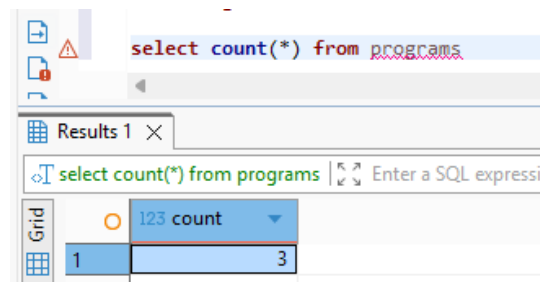
The screenshot shows a SQL query editor with the following statement:

```
insert into programs (program_name)
values
('BS in Information Technology'),
('BS in Computer Science'),
('BS in Computer Engineering')
returning id
```

Below the editor, the results are displayed in a grid titled "programs 1". The grid has two columns: "id" and "program_name". The data is as follows:

id	program_name
1	BS in Information Technology
2	BS in Computer Science
3	BS in Computer Engineering

Verify the count of the programs table



The screenshot shows a SQL query editor with the following statement:

```
select count(*) from programs
```

Below the editor, the results are displayed in a grid titled "Results 1". The grid has two columns: "count" and "program_name". The data is as follows:

count	program_name
3	

insert data in sections table

The screenshot shows a SQL IDE interface. The top pane contains the following SQL statement:

```
insert into sections (code, year, program_id)
values
('M001', 2020, 1),
('M002', 2022, 2),
('M003', 2022, 3),
('M004', 2021, 1)
returning id
```

The bottom pane shows the results of the query in a table with 2 columns: 'id' and 'count'. The table has 4 rows of data.

id	count
1	1
2	2
3	3
4	4

verify the count of sections table

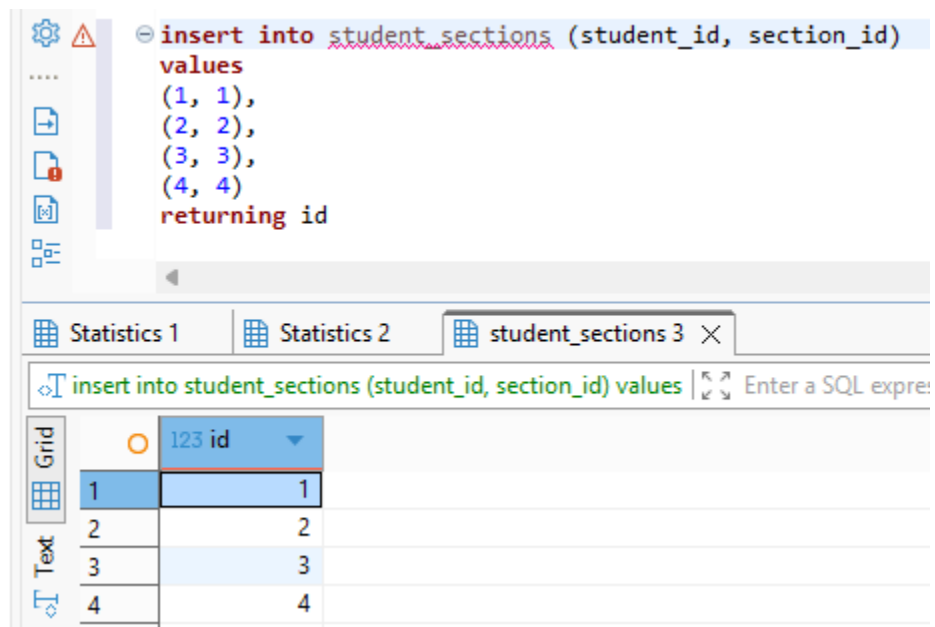
The screenshot shows a SQL IDE interface. The top pane contains the following SQL statement:

```
select count(*) from sections
```

The bottom pane shows the results of the query in a table with 2 columns: 'count' and 'id'. The table has 1 row of data.

count	id
4	1

Insert data in student_sections junction table



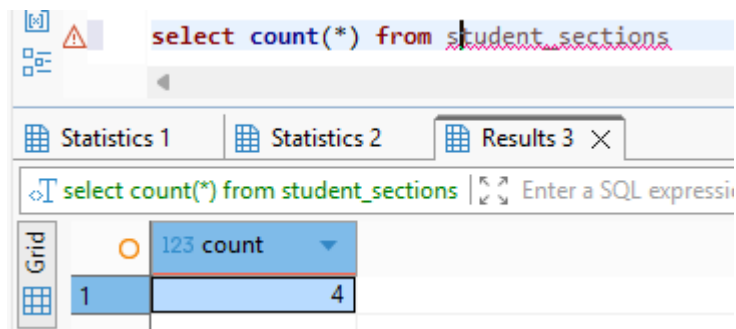
The screenshot shows a database management interface. The top panel displays an SQL insert statement:

```
insert into student_sections (student_id, section_id)
values
(1, 1),
(2, 2),
(3, 3),
(4, 4)
returning id
```

Below the SQL editor, there are tabs for "Statistics 1", "Statistics 2", and "student_sections 3". The "student_sections 3" tab is active, showing a grid of results. The grid has two columns: "id" and "123 id". The data is as follows:

	id	123 id
1	1	1
2	2	2
3	3	3
4	4	4

Verify the count of student_sections junction table



The screenshot shows a database management interface. The top panel displays an SQL count query:

```
select count(*) from student_sections
```

Below the SQL editor, there are tabs for "Statistics 1", "Statistics 2", and "Results 3". The "Results 3" tab is active, showing a grid of results. The grid has two columns: "count" and "123 count". The data is as follows:

	count	123 count
1	4	4

Insert data in student_grades

The screenshot shows a database IDE with a query editor and a results grid. The query editor contains the following SQL statement:

```
insert into student_grades (student_id, grade)
values
(1, 100),
(1, 100),
(1, 100),
(2, 90),
(2, 85),
(2, 80),
(3, 75),
(3, 75),
(3, 75),
(4, 90),
(4, 92),
(4, 95)
returning id
```

The results grid shows the following data:

id	123 id
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
12	12

Verify the count of student_grades

The screenshot shows a database IDE with a query editor and a results grid. The query editor contains the following SQL statement:

```
select count(*) from student_grades
```

The results grid shows the following data:

count	123 count
12	12

A tooltip is visible over the results grid, displaying the following information:

- Connectio
- Time: 202
- Query: sel

Part 3 – Queries

display all students with their program names

```
select s.id, s.first_name, s.last_name, s.year, s.gender, p.program_name
from students s
join student_sections ss on s.id = ss.student_id
join sections sec on ss.section_id = sec.id
join programs p on sec.program_id = p.id
```

	123 id	AZ first_name	AZ last_name	123 year	AZ gender	AZ program_name
1	1	Ron Vincent	Cada	3	male	BS in Information Technology
2	2	Mika Andrea	Gomez	2	female	BS in Computer Science
3	3	Roosc	Zano	1	male	BS in Computer Engineering
4	4	Kurt	Laja	3	male	BS in Information Technology

display students with their assigned section codes

```
select s.id, s.first_name, s.last_name, s.year, s.gender, p.program_name, sec.code
from students s
join student_sections ss on s.id = ss.student_id
join sections sec on ss.section_id = sec.id
join programs p on sec.program_id = p.id
```

	123 id	AZ first_name	AZ last_name	123 year	AZ gender	AZ program_name	AZ code
1	1	Ron Vincent	Cada	3	male	BS in Information Technology	M001
2	2	Mika Andrea	Gomez	2	female	BS in Computer Science	M002
3	3	Roosc	Zano	1	male	BS in Computer Engineering	M003
4	4	Kurt	Laja	3	male	BS in Information Technology	M004

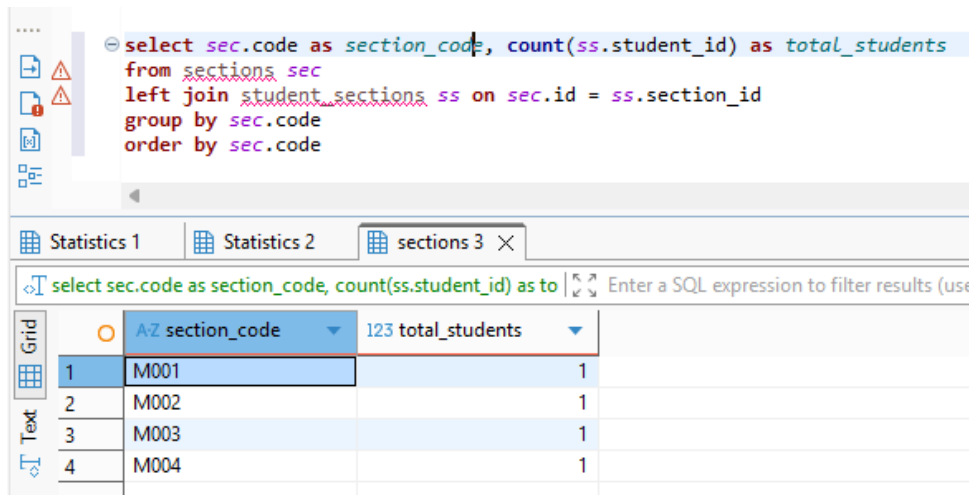
Retrieve all the currently enrolled students

In here student Roosc Zano is not shown since he is not enrolled

```
select * from students
where enrolled = true
```

	123 id	AZ first_name	AZ last_name	123 year	AZ gender	enrolled	created_at
1	1	Ron Vincent	Cada	3	male	[v]	2026-05-11 15:08:43.068 +0800
2	2	Mika Andrea	Gomez	2	female	[v]	2026-05-11 15:08:43.068 +0800
3	4	Kurt	Laja	3	male	[v]	2026-05-11 15:08:43.068 +0800

Count the number of students per section using GROUP BY



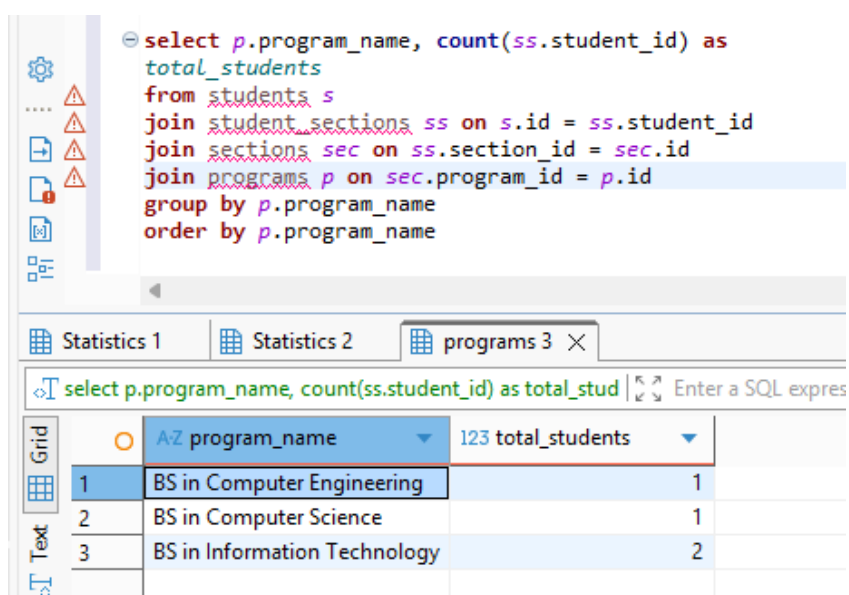
The screenshot shows a database query editor with a SQL query in the top pane and a results grid in the bottom pane. The query is:

```
select sec.code as section_code, count(ss.student_id) as total_students
from sections sec
left join student_sections ss on sec.id = ss.section_id
group by sec.code
order by sec.code
```

The results grid shows the following data:

	AZ section_code	123 total_students
1	M001	1
2	M002	1
3	M003	1
4	M004	1

Count the number of students per program



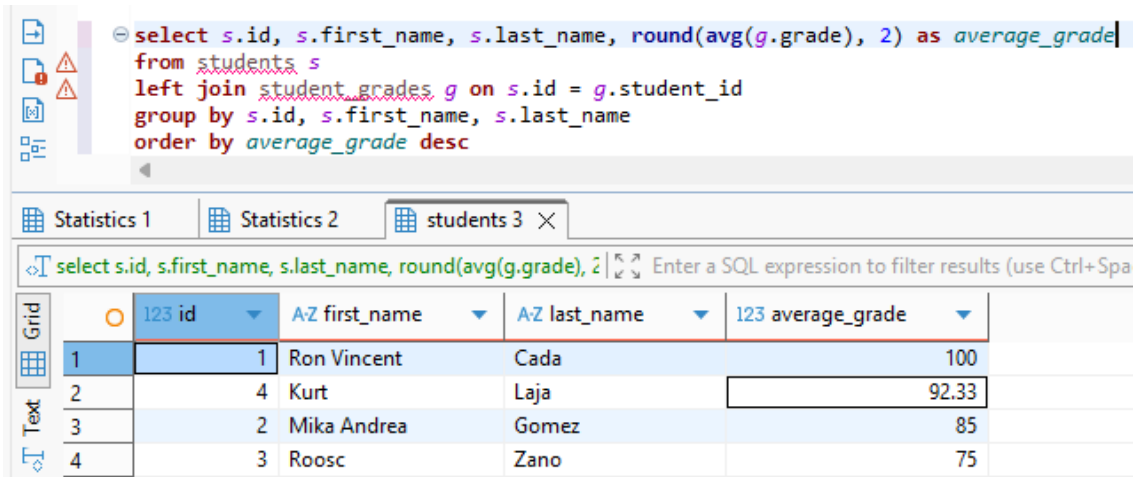
The screenshot shows a database query editor with a SQL query in the top pane and a results grid in the bottom pane. The query is:

```
select p.program_name, count(ss.student_id) as total_students
from students s
join student_sections ss on s.id = ss.student_id
join sections sec on ss.section_id = sec.id
join programs p on sec.program_id = p.id
group by p.program_name
order by p.program_name
```

The results grid shows the following data:

	AZ program_name	123 total_students
1	BS in Computer Engineering	1
2	BS in Computer Science	1
3	BS in Information Technology	2

Computing the average grade of each student using AVG



```
select s.id, s.first_name, s.last_name, round(avg(g.grade), 2) as average_grade
from students s
left join student_grades g on s.id = g.student_id
group by s.id, s.first_name, s.last_name
order by average_grade desc
```

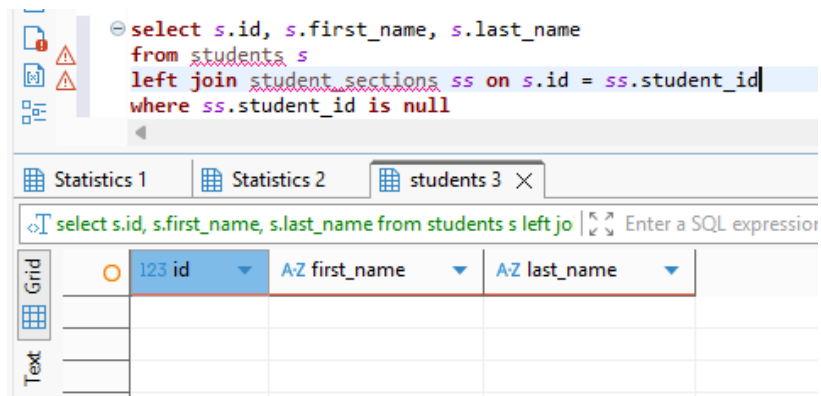
Statistics 1 | Statistics 2 | students 3 X

select s.id, s.first_name, s.last_name, round(avg(g.grade), 2) Enter a SQL expression to filter results (use Ctrl+Spa

	123 id	AZ first_name	AZ last_name	123 average_grade
1	1	Ron Vincent	Cada	100
2	4	Kurt	Laja	92.33
3	2	Mika Andrea	Gomez	85
4	3	Roosc	Zano	75

Finding students who are not assigned to any section using LEFT JOIN with IS NULL

In here there are no students found with no sections assigned



```
select s.id, s.first_name, s.last_name
from students s
left join student_sections ss on s.id = ss.student_id
where ss.student_id is null
```

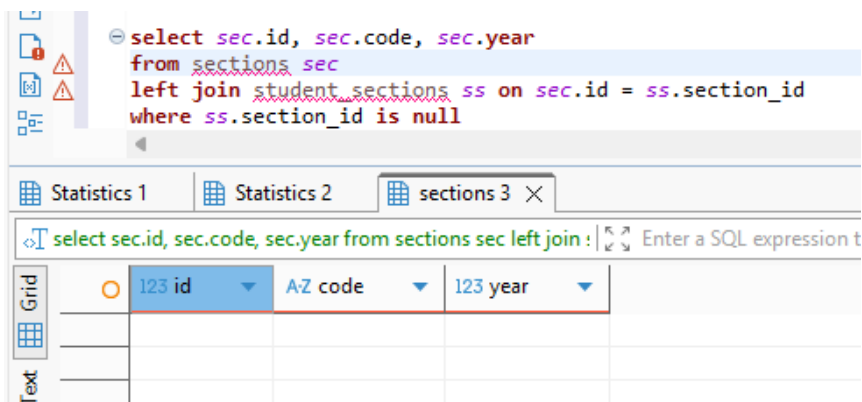
Statistics 1 | Statistics 2 | students 3 X

select s.id, s.first_name, s.last_name from students s left jo Enter a SQL expression

	123 id	AZ first_name	AZ last_name

Finding sections with no students assigned

In here there are no sections found with no students assigned



```
select sec.id, sec.code, sec.year
from sections sec
left join student_sections ss on sec.id = ss.section_id
where ss.section_id is null
```

Statistics 1 | Statistics 2 | sections 3 X

select sec.id, sec.code, sec.year from sections sec left join : Enter a SQL expression t

	123 id	AZ code	123 year

PART 4 – Indexes & Constraints

```
explain analyze
select s.first_name, s.last_name, p.program_name, sec.code as section_code, round(avg(sg.grade), 2) as average_grade
from students s
join student_sections ss on s.id = ss.student_id
join sections sec on ss.section_id = sec.id
join programs p on sec.program_id = p.id
left join student_grades sg on s.id = sg.student_id
group by s.id, s.first_name, s.last_name, p.program_name, sec.code
order by s.id
```

Before creating Indexes

AZ QUERY PLAN	
36	Buffers: shared hit=1
37	-> Hash (cost=12.30..12.30 rows=230 width=322) (actual time=0.033..0.033 rows=3.00 loops=1)
38	Buckets: 1024 Batches: 1 Memory Usage: 9kB
39	Buffers: shared hit=1
40	-> Seq Scan on programs p (cost=0.00..12.30 rows=230 width=322) (actual time=0.027..0.028 rows=
41	Buffers: shared hit=1
42	-> Sort (cost=142.54..147.64 rows=2040 width=8) (actual time=0.020..0.021 rows=12.00 loops=1)
43	Sort Key: sg.student_id
44	Sort Method: quicksort Memory: 25kB
45	Buffers: shared hit=1
46	-> Seq Scan on student_grades sg (cost=0.00..30.40 rows=2040 width=8) (actual time=0.012..0.013 rows=1
47	Buffers: shared hit=1
48	Planning Time: 0.833 ms
49	Execution Time: 0.425 ms

create indexes

<pre>create index idx_sections_program_id on sections(program_id); create index idx_student_sections_student_id on student_sections(student_id); create index idx_student_sections_section_id on student_sections(section_id); create index idx_student_grades_student_id on student_grades(student_id);</pre>	
Statistics 1 Statistics 2 Statistics 3 Statistics 4 X	
Name	Value
Updated Rows	0
Execute time	0.009s
Start time	Mon May 11 16:07:04 PST 2026
Finish time	Mon May 11 16:07:04 PST 2026
Query	create index idx_sections_program_id on sections(program_id); create index idx_student_sections_student_id on student_sections(student_id); create index idx_student_sections_section_id on student_sections(section_id); create index idx_student_grades_student_id on student_grades(student_id)

After creating indexes

explain analyze select s.first_name, s.last_name, p.program Enter a SQL expression to filter results (use Ctrl+Space)	
Grid Text Record	A-Z QUERY PLAN
	32 Buffers: shared hit=2
	33 -> Hash Join (cost=1.09..14.29 rows=4 width=440) (actual time=0.028..0.030 rows=4.00 loops=1)
	34 Hash Cond: (p.id = sec.program_id)
	35 Buffers: shared hit=2
	36 -> Seq Scan on programs p (cost=0.00..12.30 rows=230 width=322) (actual time=0.009..0.009 row
	37 Buffers: shared hit=1
	38 -> Hash (cost=1.04..1.04 rows=4 width=126) (actual time=0.010..0.010 rows=4.00 loops=1)
	39 Buckets: 1024 Batches: 1 Memory Usage: 9kB
	40 Buffers: shared hit=1
	41 -> Seq Scan on sections sec (cost=0.00..1.04 rows=4 width=126) (actual time=0.006..0.007 row
	42 Buffers: shared hit=1
	43 Planning:
	44 Buffers: shared hit=71 read=4
	45 Planning Time: 4.976 ms
	46 Execution Time: 0.273 ms